capital

Custom SBOM Operation Development

Software Version 2015.1

February 2016

© 2016 **Mentor Graphics Corporation**
All rights reserved.

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in w hole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS CORPORATION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

RESTRICTED RIGHTS LEGEND 03/97

U.S. Government Restricted Rights. The SOFTWARE and documentation have been developed entirely at private expense and are commercial computer software provided with restricted rights. Use, duplication or disclosure by the U.S. Government or a U.S. Government subcontractor is subject to the restrictions set forth in the license agreement provided with the software pursuant to DFARS 227.7202- 3(a) or as set forth in subparagraph (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clause at FAR 52.227-19, as applicable.

Contractor/manufacturer is:
Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777.
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: www.mentor.com/supportnet
Contact Your Technical Writer: www.mentor.com/supportnet/documentation/reply_form.cfm

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other third parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the respective third-party owner. The use herein of a third- party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at: www.mentor.com/terms_conditions/trademarks.cfm.

End-User License Agreement: You can print a copy of the End-User License Agreement from: www.mentor.com/terms_conditions/enduser.cfm.

Table Of Contents

Introduction **2**
Operation selection logic 2
Additional Documentation 5
Interfaces **6**
IXOperation 6
IXOperationParametersSerializer 8
IXOperationUI 9
IXSBOMContext 10
IXSubAssembly 11
Exercise **13**
Guidance **21**
Examples **22**

Introduction

This document describes the process for developing Custom SBOM Operation plugins for the Capital Harness MPM application. This allows you to provide your own operation templates to be used for SBOM generation. If you are **not** interested in developing Custom SBOM Operations, you do not have to read this document. If you are interested in developing Custom SBOM Operations, ensure that you have read and completed the exercises in the PluginDevelopment.pdf document before reading and completing the exercises in this document.

SBOM generation relies on operations to select and group input sub-assemblies into the resulting parent sub-assembly: this is referred to as the *operation selection logic*.
The application provides the user with a list of operation templates (which will contain your plugins when you provide some). The main responsibility of a template is to provide the selection logic for the operation. The selection logic can be influenced by a number of parameters which the template also has the responsibility for defining, exposing (to the user) and managing.
The user will create operation instances – initially from templates – by placing them within an SBOM pattern. An operation instance only exists within a pattern and holds values for each parameter, which can be modified through the user interface provided by the template. Parameter values can then be stored in the database, using the serialization method provided by the template (same for deserialization and retrieval).

This is a summary of the operation template (plugin) responsibilities:

- Provide the operation selection logic
- Provide a GUI for the user to edit the operation parameters (if any)
- Provide the serialization and deserialization methods for persistence and retrieval of the operation parameters (if any)

Operation selection logic

This is the crucial part of your implementation and requires the following minimum understanding of the SBOM Generation Engine mechanisms.

The SBOM Generation Engine behaves like a *Rule Engine* in which the rules would be represented by the operation instances within the input pattern (the *rule base*). The *facts* are represented by the sub-assemblies present in the SBOM - i.e. at any given stage of the generation process, the SBOM is the fact base. Initially, the engine populates the SBOM with *leaf sub-assemblies*: there is one leaf sub-assembly per raw object (a raw object is a harness object which has an entry in the regular harness BOM). In this initial state, leaf sub-assemblies are also *root sub-assemblies*, i.e. they have no parent other than the SBOM itself. At any stage, the fact base consists in the set of root sub-assemblies within the SBOM.
Then inference is started by invoking the *engaged* operation *selector method* (which implements the operation selection logic), passing it each fact as an input (root) sub-assembly.

The operation plugin implementation selector method must provide 2 outputs:

- The “fired” flag as its Boolean return value, to indicate if the input fact is selected or not
- If fired, the set of combined facts which are the other root sub-assemblies to be combined with the input root sub-assembly in order to form the output parent sub-assembly – note that once the engine creates this new output (root) sub-assembly, it contains the input sub-assembly as well as the combined sub-assemblies as its children and therefore those are no longer root sub-assemblies / facts: the operation has consumed facts to create new ones which will then be processed in the next iteration of the fact base.

To perform this selection logic, the operation implementation is passed 2 objects to access the data it needs:

- The input root sub-assembly / fact which implements the IXSubAssembly interface. This interface allows to navigate sub-assembly relationships (parent/children) and also to retrieve any contained raw object which then allows to navigate harness relationships, using the standard business objects API (see Additional Documentation below).
- A context object which implements the IXSBOMContext interface. This interface is principally useful to retrieve the root sub-assembly / fact corresponding to a given raw object. This will be required to return the combined facts (mostly resulting from harness object navigation) as root sub-assemblies.

A typical implementation will contain the following steps:

- **Object type filtering**: operations usually apply to a main object type (even if the input sub-assembly may contain multiple object types) – e.g. a Wire Cutting operation will only consider wires and reject any input sub-assembly which doesn’t contain one.
- **Process filtering**: some operations will only apply to sub-assemblies which themselves result from prior specific operations – e.g. a Wire Terminating operation will only apply to wire sub-assemblies which result from a Wire Stripping operation. Note that this filter can make use of the type of the generator operation associated with any existing sub-assembly to make assumptions as to what process this sub-assembly has gone through so far, but this method is not recommended as it creates a brittle coupling between operations. Instead, the operation should base this filter on the structure of input sub-assemblies.
- **Harness filtering and combining**: finally, the operation can drill further down into the input model (the harness) to filter specific objects based on their relationships within the harness – e.g. a Wire Terminating Both Ends operation will select a given wire only if it doesn’t go through an unslit tube, and it has a terminal at both ends. In this step also, the operation will select the objects to combine with the input sub-assembly and return as output – e.g. for the Wire Terminating Both Ends operation, select both wire end terminals.

Note that each of these steps can be conditioned by the operation parameter values.

In this transient state, the fact base is iterated multiple times for each operation, until:

- the SBOM reaches the completion state – i.e. it contains a single root sub-assembly
- OR an error condition is detected
- OR facts are exhausted – i.e. the last iteration didn't consume or create any fact as the operation didn't fire: in this case the next operation (according to the order defined in the pattern) is engaged and iterations of the fact base are resumed for this new operation.

Once all operations have been processed as described above, the SBOM can no longer change and is in one of the following states:

- **TRANSIENT**: the SBOM contains more than 1 root sub-assemblies, because of exhaustion or error, which indicates an incomplete or incorrectly ordered pattern.
- **ERROR**: an error condition has been detected and generation had to be interrupted – this generally means that this state is also TRANSIENT.
- **COMPLETE**: the SBOM contains a single root sub-assembly which represents the complete buildable harness.

As mentioned above, error conditions can be reached during generation, caused by incorrect operation implementations and resulting in violation of the following SBOM Generation Engine design principles:

- **Convergence: Each iteration should result in fact base reduction so that the process tends to completion (only one fact left).**

  This rule will be typically violated by a greedy operation – i.e. an operation which fires unconditionally and doesn't provide any combined objects. This results after one iteration in one parent sub-assembly per raw object, then each subsequent iteration adding one parent per child (such iterations are called iso-iterations as they don't change the fact base size): this can never converge towards a single parent and the generation process would be endless if the condition was not captured by the engine counting subsequent iso-iterations and aborting when this number reaches 3.

- **Stability: Any 2 generations using the exact same input data (harness and pattern) should result in the exact same SBOMs – i.e. all sub-assemblies have the same content.**

  This rule cannot be strictly enforced by the engine as it would need to run multiple generations in random order of the fact base to verify, which would be prohibitive performance-wise. However, to respect it, operation implementations need to strictly avoid arbitrary selection – and resort to harness object name ordering when nothing else can discriminate 2 objects. See Examples at the end of this document for more information on this.

- **Tree structure: the SBOM is a strict tree structure, i.e. any node can only belong to only one parent.**

  Another possible consequence of arbitrary selection (see previous rule) is self-intersection, which occurs when the same operation selects the same object to combine with different input facts – which would result in this object having 2 different parents.

- **Rule based: operations should only deal with facts (i.e. root sub-assemblies): their input is a fact and so should their output (combined objects) be.**

  The operation implementation will respect this rule by the consistent use of the `IXSBOMContext.getOwnerRootSubAssembly()` method for retrieving combined objects (as facts) and checking the result is not null – which would happen if the selected harness object is not in the fact base initially (i.e. it is not a regular harness BOM object). Note also that this rule would be violated if the operation returned a sub-assembly which is not a root sub-assembly as a combined object.

Additional Documentation

Just to remind you, the documentation for the Java interfaces (the API) can be found within the Capital installation folder in the following location:

doc/plugin/api/index.html

Moreover, the plugins you develop will be passed in "business" objects as parameters, from which the Capital information you require can be obtained. The relationships between these objects are expressed in a UML diagram.

The UML diagram for the business objects can be found within the Capital installation folder in the following file:

doc/plugin/api/BusinessObjectsClassDiagram.png

It is important to note that these business objects give you read-only access to the underlying object model used by Capital. You will not be able to use them to modify any object model data. Also, they do not contain graphical information, just the properties and attributes of the objects and their relationship with each other.

The UML diagram for the plugin interfaces can be found within the Capital installation folder in the following file:

doc/plugin/api/PluginObjectsClassDiagram.png

Interfaces

IXOperation

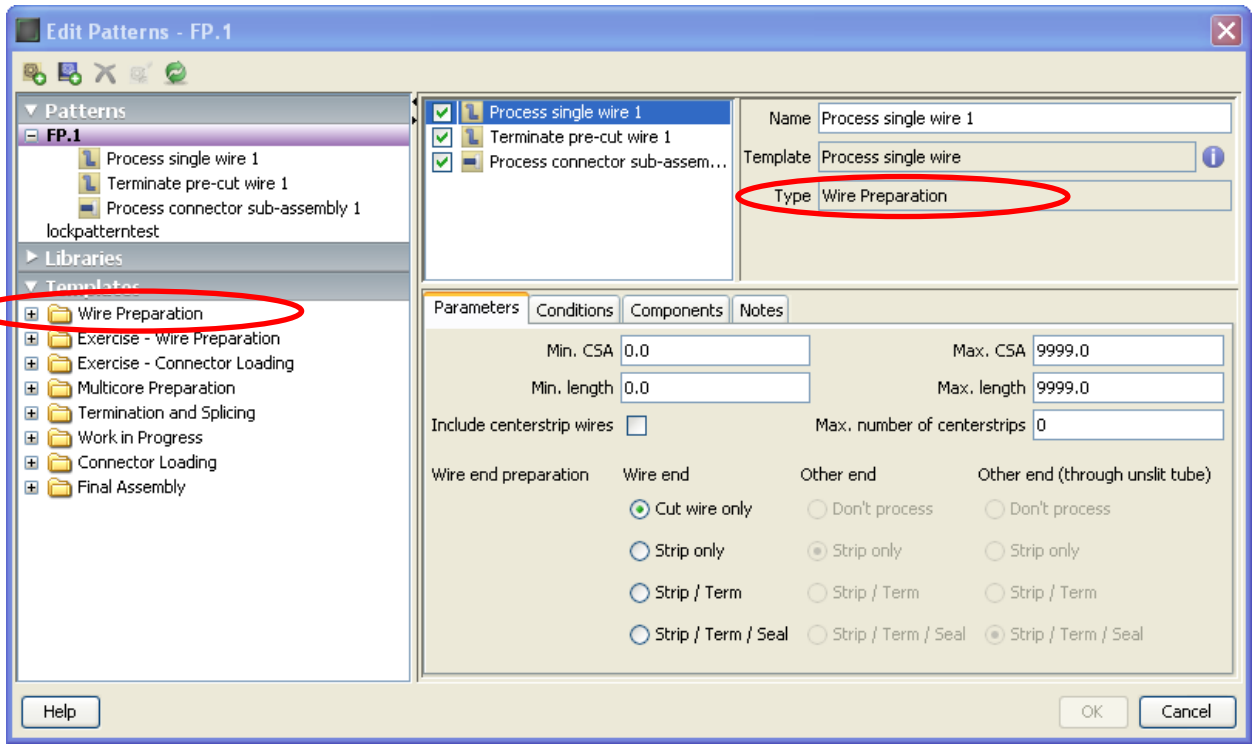
In order to build a custom SBOM Operation plugin, this is the interface that you will need to implement.

```
public interface IXOperation extends IXPlugin
{
      enum StandardOperationTypeEnum
      {

            WIRE_PREPARATION,
            MULTICORE_PREPARATION,
            TERMINATION_AND_SPLICING,
            WORK_IN_PROGRESS,
            CONNECTOR_LOADING,
            INSULATION,
            FINAL_ASSEMBLY
      }

      interface IXOperationType
      {

            String getName();

            Icon getIcon();

            short getOrderHint();

            StandardOperationTypeEnum getStandardType();
      }

      IXOperationType getType();

      int getOrderHint();

      IXOperationParametersSerilizer getParemetersSerializer();

      IXOperationUI getUI();

}
```

As you can see `IXOperation` extends the `IXPlugin` interface. This means that you will need to implement the `getName()`, `getVersion()` and `getDescription()` methods.

This interface also provides the core operation data and runtime (SBOM generation) functionality. Additionally, it also provides the build-time components, used for managing operation parameters.

`IXOperationType` – best implemented as an enum which will provide the list of available types. Type names and/or icons are only used for presentation purposes and visible in the following UI (a folder icon is displayed when the type icon is null). The order hint will determine the display order of folder types in the Templates tree:



`StandardOperationTypeEnum` – The list of exposed standard types. A standard type can be associated to the operation type to support *Process filtering* involving operations coming from different suppliers.

`IXOperationType.getStandardType()` – Retrieves the standard type associated with this operation type. This method can return a null value.

`getType()` – for this operation to return its type as an object implementing the IXOperationType interface. Please make sure the same IXOperationType object is returned each time this method is invoked, by caching this object in your operation. Note also that it is not possible to re-use an existing Capital Harness MPM operation type: your own custom operations cannot be presented within the same type folder as existing Capital Harness MPM operations. However, Capital Harness MPM operation types use the exposed standard types which can be re-used for the purpose of *Process filtering*.

`getOrderHint()` – determines the order in which operations are presented within the same type folder in the Pattern Builder dialog. If this order is unimportant to you, it is safe to return any number, this will have no effect on the operation once it is instantiated within a pattern.

`runSelector()` – The core runtime method invoked by the Generation Engine to produce a new sub-assembly. The input is a single root sub-assembly / fact as well as a context object providing various services needed to construct the output (see IXSBOMContext below). The output is a set of root sub-assembly / facts to be combined with the input for generating the parent sub-assembly and the Boolean return value which indicates whether the operation has fired (i.e. a new parent sub-assembly should be created using the set of combined objects) or not. This method implements the operation's selection logic as detailed in this manual's introduction.

`reset()` – you should implement this method if your operation implementation is stateful (which is generally not recommended) in order to reset any member variable between two executions of the generation engine. The engine invokes this methods both at the beginning and at the end of each generation.

`getParametersSerializer()` – Provides the Parameters Serializer build-time component of this operation (see IXOperationParametersSerilizer below). This method can return null if the operation has no parameter.

`getUI()` – Provides the UI build-time component of this operation (see IXOperationUI below). This method can return null if this operation has no parameter which needs exposing in the Pattern Builder dialog.

IXOperationParametersSerializer

This interface represents one of the build-time components of the operation, provided by the IXOperation.getParametersSerilizer() implementation. This component is responsible for serializing and deserializing the operation's parameters to/from string so that parameter values can be included in the application's persistence mechanism, i.e. it allows saving/loading parameters values along with an operation instance in a pattern without the application being aware of how these parameters are implemented.

```
public interface IXOperationParametersSerializer
{
      String serialize();

      void deSerialize (String parameters) throws ParseException;

}
```

`serialize()` – called by the Pattern Builder UI (see screenshot above), when the OK button is pressed, to persist the operation instance along with its parameter values. This method should construct and return a string representation of all the operation parameter values (for instance as a

comma separated list of values). This method will be generic if all operations use the same format for their parameters.

`deserialize()` – called by the Pattern Builder UI (see screenshot above), when displayed to load parameter values in the operation editors, or by the Generation Engine when invoked to ensure involved operations will use the latest parameter values. This method should parse the string provided as an input argument in order to set the operation parameter values. This method will be generic if all operations use the same format for their parameters.

IXOperationUI

This interface represents the other build-time component of the operation, provided by the IXOperation.getUI() implementation. This component provides all the UI elements related to this operation, which are used by Capital Harness MPM to present the operation in its UI and allow the user editing it in the Pattern Builder dialog.

```
public interface IXOperationUI
{
      interface IXParameterChangeListener
      {

            void valueChanged();

            void validityChanged(boolean isValid);
      }

      JPanel getParametersEditor(IXParameterChangeListener
listener);

      Icon getIcon();

      URL getInformationAsHTML();

      String getInformationAsText();
}
```

`getIcon()` – for this operation to return an icon which depicts it in the Pattern Builder GUI. This icon will also be displayed in the SBOM Browser Tree, against sub-assemblies which were generated by this operation.

`getInformationAsHTML()` – for this operation to return an HTML page containing descriptive information about the template (this information is scoped to the template as opposed to a single instance). Images and a stylesheet can be included, however hyperlinks are not supported. If a non null URL (to an HTML page) is returned, then clicking the Info button ( ) for this operation will display this page in a popup window.

`getInformationAsText()` – for this operation to return a character string containing descriptive information about the template (this information is scoped to the template as opposed to a single instance). This method is called only if the previous `getInformationAsHTML()` returns null. If a non null string is returned, then clicking the Info button ( ) for this operation will display the returned text in a popup window.

`getParametersEditor()` – for this operation to return a panel (javax.swing.JPanel) which contains the necessary widgets for the user to modify its parameter values. This panel (if not null), is displayed in the Pattern Builder GUI in the "Parameters" tag of the selected operation instance. User entries validity can be managed using the `IXParameterChangeListener` object passed as an input parameter (see below).

`IXParameterChangeListener` – you should use this interface in your implementation of the `getParametersEditor()` method if you wish to manage the validity of user input for your parameter values. To do this, your widgets should have listeners which track any user modification. Once modification is detected, you should call the `valueChanged`() method to inform the Pattern Builder dialog, so the system knows the operation has changed and will need to be persisted when the OK button is next selected. Also, you can call the `validityChanged`() method if the entry is invalid, in which case the Pattern Builder dialog will disable the OK button, or if the entry has become valid again (after being invalid), in which case the OK button is enabled. Note that any display change of your widgets in case of invalidity (e.g. red text, tooltip etc…) should be managed by you as the system is not aware of those widgets.

IXSBOMContext

An object implementing this interface is passed to the IXOperation.runSelector() method by the engine to provide context information about the generation run, and integration services required by the operation selection logic.

```
public interface IXSBOMContext
{
      IXSubAssembly getOwnerRootSubAssembly(IXConnectivityObject
harnessObject);

      <T extends IXConnectivityObject> Collection<T>
getHarnessObjectsOfType(Class<T> type);

      void logMessage(String message);

      int getCurrentEngineIterationNumber();

      void setIterationLoopingMode(boolean mode);

      boolean isIterationLoopingModeOn();
}
```

`getOwnerRootSubAssembly()` – This method must be used by all operations in order to return combined objects which comply with the engine *Rule based design principle*. As seen previously, the selection logic in its *harness filtering and combining* step will select harness objects to combine with the input sub-assembly. However, the engine is not able to process harness objects as such, but only root sub-assemblies / facts. Therefore, this method must be used to retrieve the root sub-assembly / fact containing the selected harness object. If null is returned, it means the selected object is not part of the fact base and shouldn't be returned – i.e. the object cannot be part of an SBOM sub-assembly.

`getHarnessObjectsOfType()` – This utility method allows to interrogate the fact base to find harness objects of a given type, by scanning leaf sub-assemblies.

`logMessage()` – Can be used to output operation specific messages to the SBOM console.

`getCurrentEngineIterationNumber()` – Provides the fact base iteration number for this operation: each operation is called repeatedly for the whole fact base in one iteration. If at the end of this iteration no fact has been consumed (and the SBOM is not yet COMPLETE), i.e. the state of the SBOM is EXHAUSTED, then the next operation is engaged and iterations start again from 0 for this new operation. Otherwise, the operation remains engaged and a new iteration occurs with the next incremented number. It is recommended to avoid using this method to support operation selection logic as it creates a brittle coupling between the operation and the engine implementation.

`setIterationLoopingMode()` – By default, the engine will consider an operation exhausted when it has not fired during the last iteration of the fact base. In this case, the engine will engage the next operation. In some cases, the operation might use an iteration to set up some state and not fire, but needs to be invoked again in the next iteration to use this state and fire accordingly. By setting the iteration looping mode, this operation can instruct the engine to proceed to the next iteration of the fact base even if no firing occurred during the last iteration.
It is strongly recommended not to use this method and to provide state independent operations instead.

`isIterationLoopingModeOn()` – Used by the engine to retrieve the looping mode which may have been set by an operation, so that it will continue iterations even if the operation has not fired in the last iteration.

IXSubAssembly

This interface is implemented by every object in the fact base and is used by the operation to analyze SBOM sub-structures passed to its selector by the engine, usually in the object type filtering and process filtering steps.
Note also that this interface can be used in custom actions to read the SBOM structure – for presentation or reporting purposes.

```
public interface IXSubAssembly extends IXWriteableObject
{
      IXOperation.IXOperationType getGeneratorOperationType();

      IXSubAssembly getParent();

      IXSBOM getSBOM();

      Collection<? extends IXSubAssembly> getChildren();

      IXConnectivityObject getHarnessObject();

      IXLibraryObject getOperationComponent();

}
```

`getGeneratorOperationType()` – Provides the type of the operation which was used to generate this sub-assembly. Note that this may be null if the SBOM input pattern is not loaded, which would typically occur in a non SBOM application context (e.g. a custom action running in Capital HarnessXC) or if the SBOM-pattern link was broken (for instance after importing the SBOM from an XML file). As mentioned previously, it is recommended not to use this method to support operation selection logic.

`getParent()` – Provides the parent sub-assembly for this sub-assembly, which is null if this sub-assembly is a root sub-assembly.

`getSBOM()` – Provides the SBOM containing this sub-assembly.

`getChildren()` – Provides the children sub-assemblies of this sub-assembly. A leaf sub-assembly will have no children (empty list). The order of the returned list is stable across multiple generation of the same SBOM.

`getHarnessObject()` – Provides the harness (raw) object associated to a leaf sub-assembly. This will be null if the sub-assembly is not a leaf or if the harness object is not loaded. Therefore, the correct test to check if a sub-assembly is a leaf is `getChildren().isEmpty()` as opposed to `getHarnessObject() != null`.

`getOperationComponent()` – Returns additional components provided by the generator operation. Such components do not exist in the fact base nor in the design, but can be injected by the operation into a sub-assembly. For instance, a piece of tearable tape to hold some coiled wires would constitute such a component.

Exercise

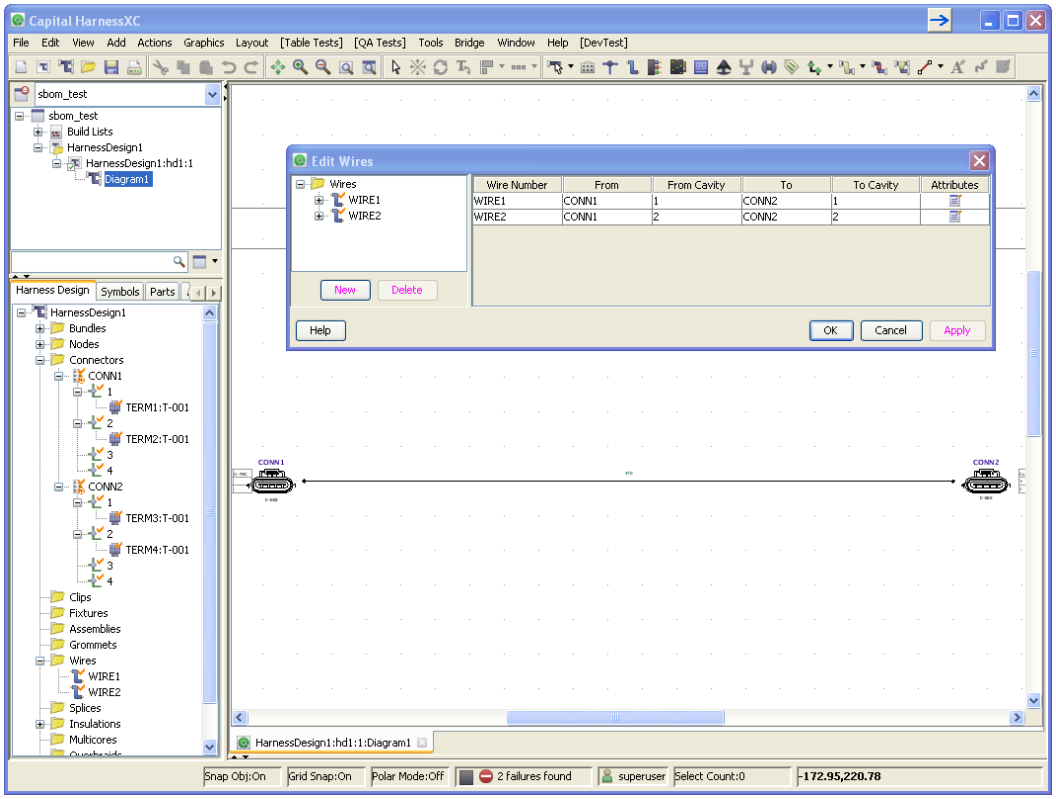
In this exercise, you will develop a custom SBOM operation for cutting, stripping and terminating wires.

You need to be familiar with Capital HarnessXC to do this exercise.

1) First, you should start by creating some test data.

   a. Start Capital HarnessXC

   b. Create a new design. There should be two connectors and two wires linking them as in the screenshot below:



   c. Save your new design and exit Capital HarnessXC.

2) The next step is to create a new SBOM operation template which will cut, strip and terminate the wires.

a. Ensure that your Java development environment has been set-up. If you have completed the Java plugin exercise then it should already be setup. If you have not, then you should stop this exercise and complete it now.

b. Open a UNIX/DOS console. Ensure that the environment variables required for Java, ANT and Capital have been setup in this console. cd into the following folder within the Capital installation:

```
doc/plugin/examples/Java
```

c. In the above folder, there is an existing template operation plugin for cutting and terminating wires. Here is the location of the file as seen from the Capital installation folder:

```
doc/plugin/examples/Java/src/com/example/plugin/sbom/CutAndS
tripWireOperation.java
```

Create a new folder in the following location within the Capital installation folder:

```
doc/plugin/examples/Java/src/exercise
```

The folder `src` is the directory where all the example Java plugin source code has been placed. This should exist already. The new folder `exercise` is the package in which your plugin will reside.

d. Copy the template into the new folder you have just created and change its name to `CutStripAndTerminateWireOperation.java`. The result should be a file in the following location:

```
doc/plugin/examples/Java/src/exercise/CutStripAndTerminateWi
reOperation.java
```

e. Edit this new file.

f. Ensure the plugin is in the correct package. This can be done by changing the following line at the start of the file (before the imports):

```
package exercise;
```

instead of:

```
package com.example.plugin.sbom;
```

If you use a template as a starting point for your new plugin, you must remember to specify the package it is in.

g. This operation uses a superclass: AbstractOperation which provides default implementations for certain IXOperation methods. Since your new class is no longer in the same package but still extends this AbstractOperation class, you should add an import statement for it as follows:

```
import com.example.plugin.sbom.AbstractOperation;
```

h. You also need to ensure that the name of the class matches the name of the file. You will need to replace `CutAndStripWireOperation` with `CutStripAndTerminateWireOperation` in this file. There are two places where this needs to be done. One is the class definition, the other is the constructor.

i. Change the name and description returned by the plugin by changing the constructor method as follows:

```
public CutStripAndTerminateWireOperation()
{
      super ("Exercise operation: Cut, strip and terminate
wires",
            "1.0",
            "Cut, strip and terminate wire at both ends");
}
```

j. Remember to save the file when these changes are completed.

k. Back at the UNIX/DOS console, ensure that you are in the following folder:

```
doc/plugin/examples/Java
```

l. Build your new plugin by typing the following command:

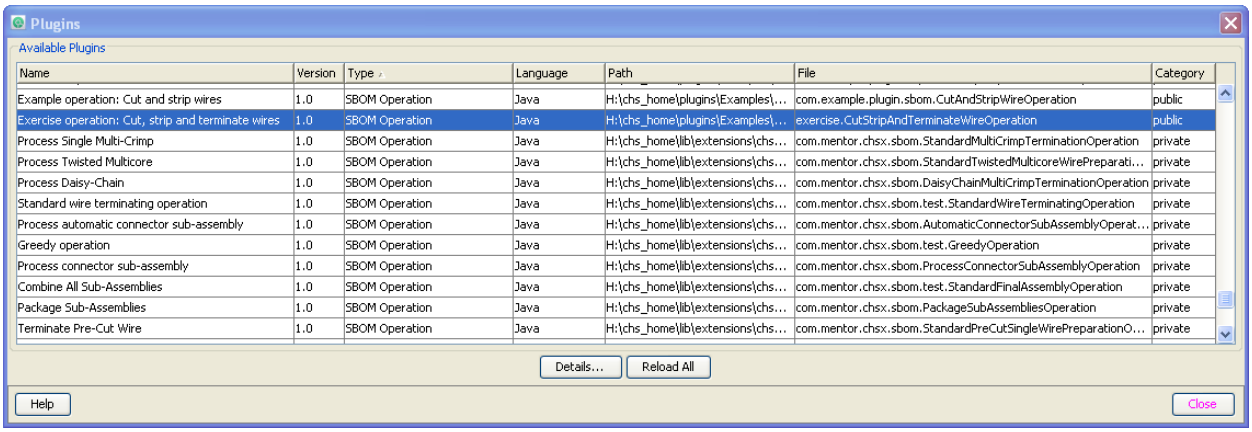
```
ant
```

m. If everything is fine, you will eventually get a "BUILD SUCCESSFUL" message. If you do not, you will need to check the environment variables or check for (and fix) errors in the `CutStripAndTerminateWireOperation.java` file and try again.

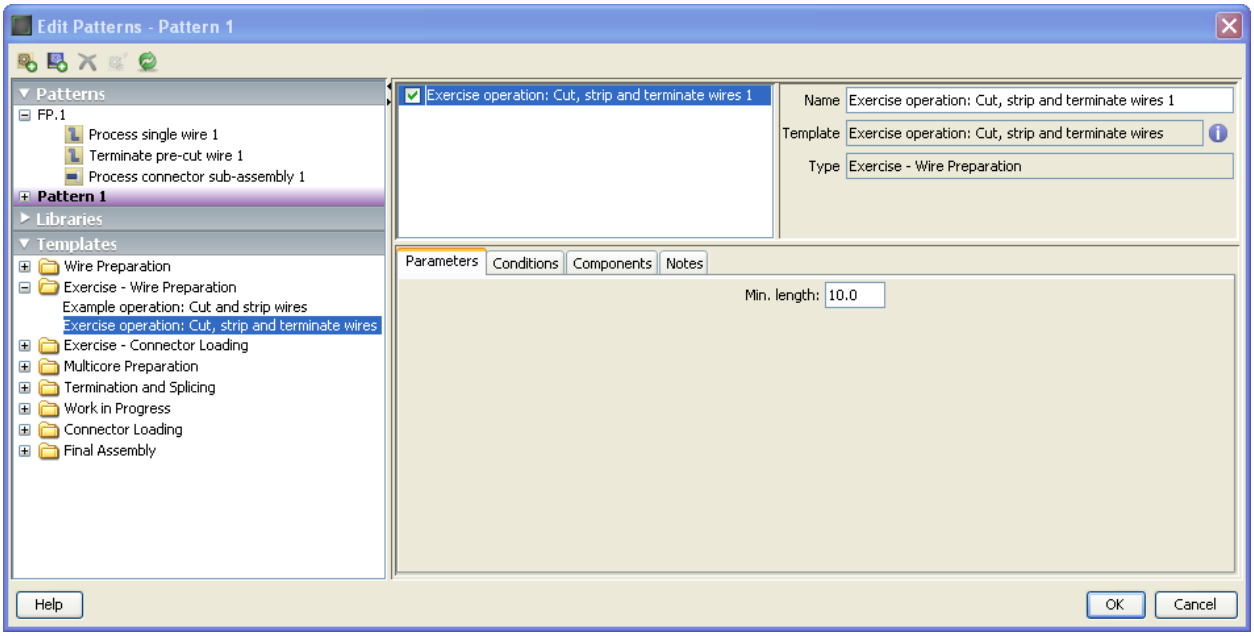
n. Start the Capital Harness MPM application.

o. From the main menu, click on the File -> Plugins… menu

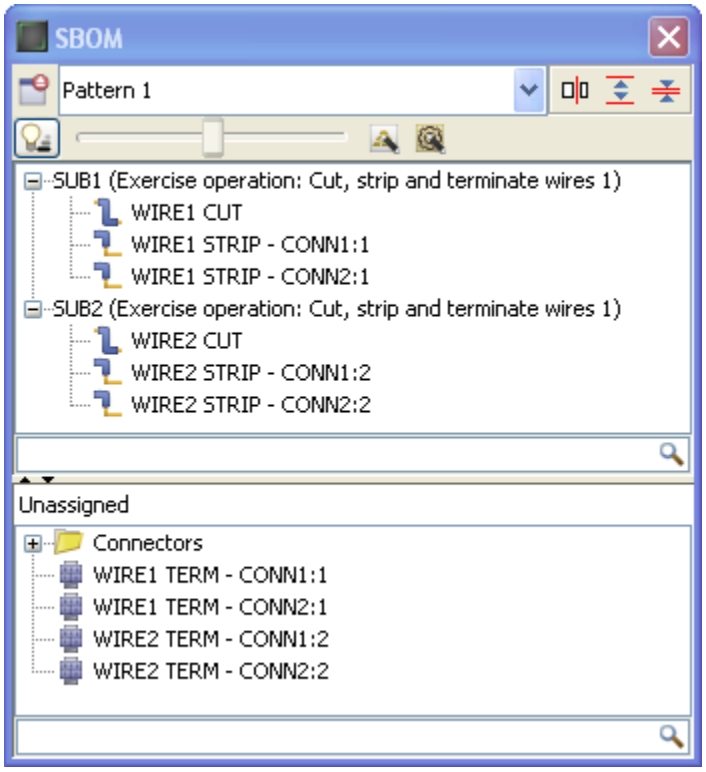
p. Your new operation template should be visible following the template you copied it from, and followed by built-in operation templates provided as private plugins:



q. Open the Pattern Builder dialog and create a pattern which contains just this new operation:



r. Open your project, right-click on the design and select the “New SBOM…” menu item and select the pattern just created. You will obtain the following SBOM:



Note at this stage that this operation only cuts and strips the wires, leaving terminal and connectors unassigned.

3) At this point you have a new plugin. However, it has not been modified to implement the inclusion of terminals. This is your next objective.

   a. Edit the `CutStripAndTerminateWireOperation.java` file.

   b. In the runSelector() method, **uncomment** the following code:

```
      @Override
      public boolean runSelector(IXSubAssembly subAssembly,
IXSBOMContext context,
                  Set<IXSubAssembly> combinedObjects)
      {
            IXObject harnessObj =
subAssembly.getHarnessObject();
            context.logMessage("Invoked Operation: " +
getName() + "- harness obj = " + harnessObj);

            //
            // ========> Object type and process filtering
steps:
```

```
            // - only look at leaf wires
            // i.e. wires not already consumed by another sub-
assembly
            if (harnessObj instanceof IXWire) {
                  IXWire wire = (IXWire) harnessObj;

                  //
                  // ========> Harness filtering and combining
                  // - conditionned by minimum length parameter
                  // - select wire ends and single-crimp
terminals
                  double length =
Double.parseDouble(wire.getAttribute("ModifiedLength"));

                  if (length >= minLengthValue) {
                        // Strip by combining with the wire
ends
                        for (IXWireEnd wireEnd :
wire.getWireEnds()) {

      combinedObjects.add(context.getOwnerRootSubAssembly(wire
End));
                        }

                        // Terminate by combining with the
terminals - don't include multi-crimp terminals
                        for (IXTerminal terminal :
getWireSingleTerminals(wire)) {

      combinedObjects.add(context.getOwnerRootSubAssembly(term
inal));
                        }

                        // Operation fires for this wire -
implicitely means the wire is cut
                        return true;
                  }
            }

            return false;
      }
```

Note the use of the `getWireSingleTerminals()` private method call to retrieve the end terminals of the wire.

c. Also change the version number of your plugin by **modifying** the constructor as below:

```
    public CutStripAndTerminateWireOperation()
    {
          super ("Exercise operation: Cut, strip and terminate
wires",
                "1.1",
                "Cut, strip and terminate wire at both ends");
    }
```

d. Save the file.

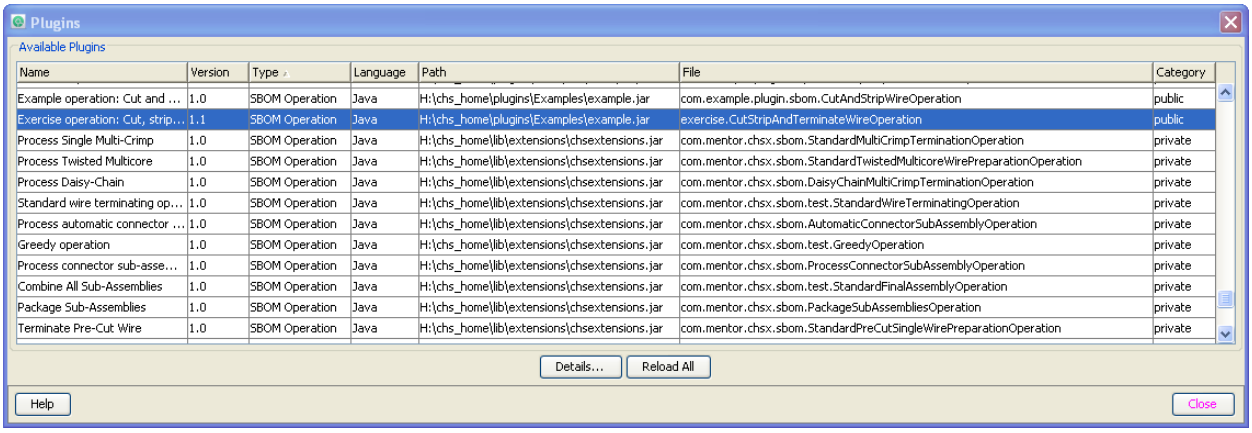
e. Rebuild your plugin by typing the following command in the DOS/UNIX console:

```
ant
```

f. If everything is fine, you will eventually get a "BUILD SUCCESSFUL" message. If you do not, you will need to check for (and fix) errors in the `CutStripAndTerminateWireOperation.java` file and try again.

g. Go back to Capital Harness MPM.

h. Press the [Reload All] button on the Plugins dialog. You will see the version number of your new plugin change:



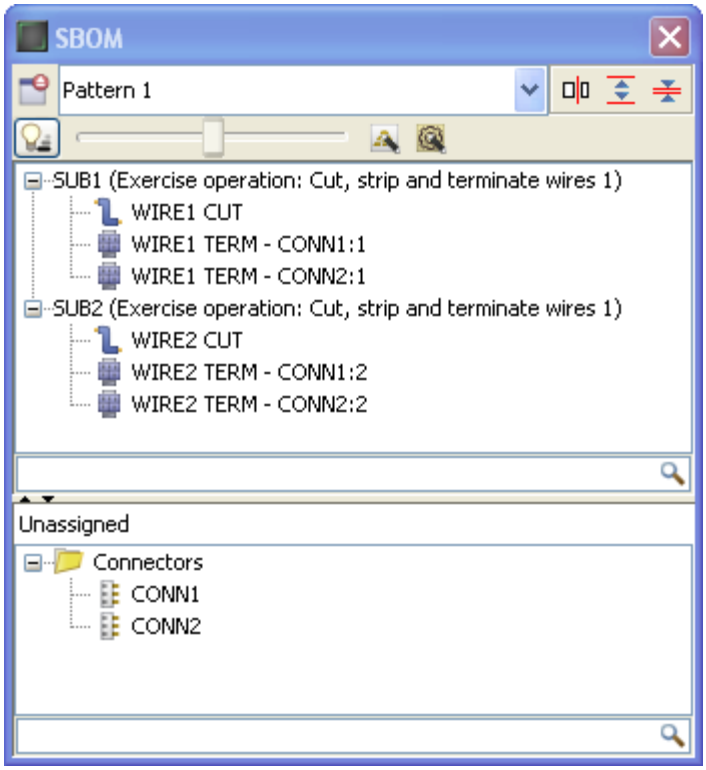
Name	Version	Type	Language	Path	File	Category
Example operation: Cut and ...	1.0	SBOM Operation	Java	H:\chs_home\plugins\Examples\example.jar	com.example.plugin.sbom.CutAndStripWireOperation	public
Exercise operation: Cut, strip...	1.1	SBOM Operation	Java	H:\chs_home\plugins\Examples\example.jar	exercise.CutStripAndTerminateWireOperation	public
Process Single Multi-Crimp	1.0	SBOM Operation	Java	H:\chs_home\lib\extensions\chsextensions.jar	com.mentor.chsx.sbom.StandardMultiCrimpTerminationOperation	private
Process Twisted Multicore	1.0	SBOM Operation	Java	H:\chs_home\lib\extensions\chsextensions.jar	com.mentor.chsx.sbom.StandardTwistedMulticoreWirePreparationOperation	private
Process Daisy-Chain	1.0	SBOM Operation	Java	H:\chs_home\lib\extensions\chsextensions.jar	com.mentor.chsx.sbom.DaisyChainMultiCrimpTerminationOperation	private
Standard wire terminating op...	1.0	SBOM Operation	Java	H:\chs_home\lib\extensions\chsextensions.jar	com.mentor.chsx.sbom.test.StandardWireTerminatingOperation	private
Process automatic connector ...	1.0	SBOM Operation	Java	H:\chs_home\lib\extensions\chsextensions.jar	com.mentor.chsx.sbom.AutomaticConnectorSubAssemblyOperation	private
Greedy operation	1.0	SBOM Operation	Java	H:\chs_home\lib\extensions\chsextensions.jar	com.mentor.chsx.sbom.test.GreedyOperation	private
Process connector sub-asse...	1.0	SBOM Operation	Java	H:\chs_home\lib\extensions\chsextensions.jar	com.mentor.chsx.sbom.ProcessConnectorSubAssemblyOperation	private
Combine All Sub-Assemblies	1.0	SBOM Operation	Java	H:\chs_home\lib\extensions\chsextensions.jar	com.mentor.chsx.sbom.test.StandardFinalAssemblyOperation	private
Package Sub-Assemblies	1.0	SBOM Operation	Java	H:\chs_home\lib\extensions\chsextensions.jar	com.mentor.chsx.sbom.PackageSubAssembliesOperation	private
Terminate Pre-Cut Wire	1.0	SBOM Operation	Java	H:\chs_home\lib\extensions\chsextensions.jar	com.mentor.chsx.sbom.StandardPreCutSingleWirePreparationOperation	private

The change in version number confirms that your new plugin has been loaded.

i. Close the Plugins dialog.

4) At this point, you are in a position to try your new plugin.

a. In the open SBOM browser window, click the Regenerate button (). Note the change in the new SBOM structure: terminals have now been removed from the Unassigned area and are included into the wire sub-assemblies. Note that strip objects are still present but not displayed by the browser because corresponding terminal are present:



At this point, you should know the following:

- How to iteratively develop Java SBOM operation templates.
- How to interrogate the business objects in Java.

Guidance

Always develop SBOM operation templates in Java for performance. Also ensure that any debug messages are removed or disabled before your custom constraint is deployed.

If you have developed your own operation templates, and they are in use at your company, care should be taken when maintaining those templates. You should avoid renaming the class or moving the class to another package. If you do this, any previously created SBOM pattern will no longer be able to find the operation template if you deploy an updated plugin that has a new class name or package. Any operation instance used in a pattern will have a reference to that template. This reference will include the class and package name for that template.

Examples

At this point, you should have a good idea how to implement this type of plugin. Some examples have been provided to help you. These examples can be found in the following location within the Capital installation:

```
doc/plugin/examples/Java/src/com/example/plugin/sbom
```

The following examples are good starting points for Capital Harness MPM:

- `CutAndStripWireOperation.java` – this example shows the basics of operation selection logic as well as operation parameter management.

- `ConnectorFeedingOperation.java` – this example shows how to process networked objects (connectors) in a constant order so that the SBOM engine Stability design principle is met.

You can copy and modify the examples to implement your own plugins if you wish. However, you need to check the license and copyright information at the start of the files before you do this.

The examples provided may be different in future releases of Capital. If you do use these examples, therefore, you need make a copy of them (we reserve the right to add, remove or modify these examples).